F1 Podium Prediction

Milestone 1 Report

Team	Team 28
Members (name - ID - tutorial)	Ali Abuelkheir - 58-3693 - T17 Maria Awad - 58-5927 - T28 Rodaina Elsawy - 58-16542 - T24 Monica Samaan - 58-4467 - T18
Notebook	ms1_v2.ipynb (Kaggle link and GitHub link on the submission form)
Dataset	Formula 1 World Championship (1950-2024), Kaggle / Ergast, 14 linked CSV tables

1. Task

For every driver entering a race we predict whether the driver finishes on the podium (**positionOrder <= 3**). One row is one driver in one race, so the key (raceId, driverId) must be unique. The prediction is made **after qualifying and before the race starts**. Everything the race itself produces (finishing position, points, laps, status, pit stops, lap times, and the standings after the race) is **leakage** and is never used as a feature.

Podiums are rare: 13.7% of the final rows (about 3 of 20 drivers in a modern race, far fewer in eras with 30+ entrants). This class imbalance drives the choice of metric in section 6.

2. Audit and cleaning

2.1 Audit

- **Formatting:** in our copy, constructor_results and constructor_standings have spaces and quotes around column names and text (" raceId"). Merges would fail, so they are stripped first.
- **Sentinels:** missing values are written as the text \N in 6 tables (for example results.position, qualifying.q2/q3, drivers.number). Pandas reads them as text, so they are counted per column (Figure 1), replaced by NaN, and counted again (0 left). Some cells are truly empty, which pandas already reads as NaN.
- **Primary keys:** all 14 tables have a unique, non-null primary key.
- **Foreign keys:** 23 links checked (for example results.raceId -> races, qualifying.driverId -> drivers): 0 orphans.
- **Modeling key:** (raceId, driverId) is not unique in results.csv: 85 pairs, 91 extra rows (section 2.3).

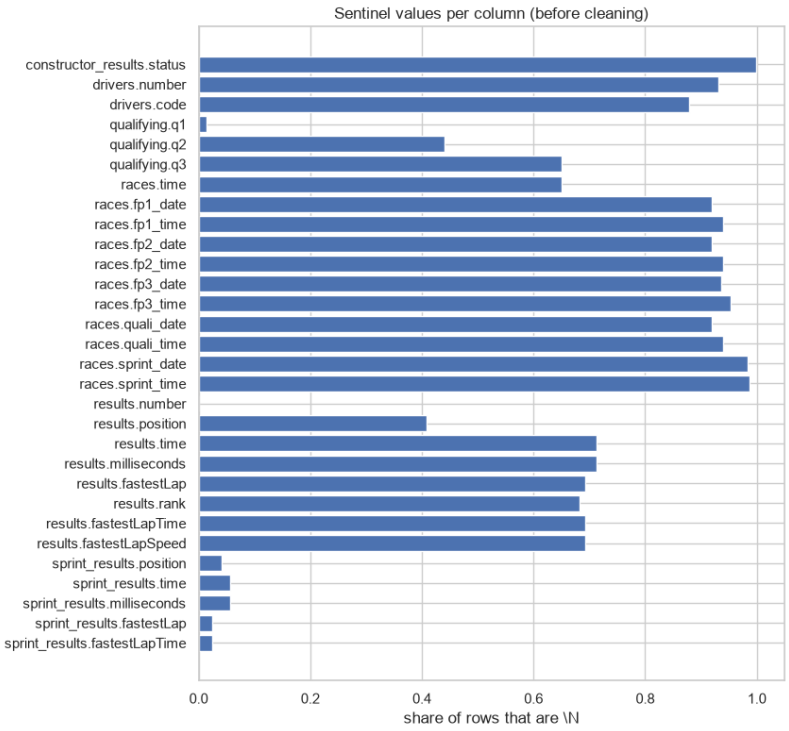


Figure 1. Share of \N sentinel values per column, before cleaning.

2.2 Semantic typing

- Six results columns (number, position, milliseconds, fastestLap, rank, fastestLapSpeed) to numbers. races.date and drivers.dob to dates.
- Qualifying lap times ("1:26.572") to seconds. One lap of 16:42.640 (1002.6 s) is far from every other value (the next is 153.9 s), so it is set to missing. The other slow laps fit a wet session and are kept.
- The other duration columns are post-race and never used (results.time, results.fastestLapTime, lap_times.time, pit_stops.duration). Race, lap and pit-stop times already have a numeric milliseconds column next to the text, so the text copies are left as they are.
- One driver nationality had a stray space. circuits.country uses both "USA" and "United States", so they are merged.

2.3 Grain: one row per (raceId, driverId)

Almost all duplicates come from 1950-1964 **shared drives**: a driver could take over a teammate's car during the race, which gives one row per car. **Rule: keep the row with the lowest positionOrder.** The row answers "did this driver finish on the podium?", and a podium in the shared car is an official podium for that driver. Limitation: the kept row can carry the grid slot of the car the driver took over, because the data does not say which car the driver started in. The opposite case, **two drivers sharing one car**, is kept: both have one row and both were officially on the podium (18 races have 4 or 5 podium rows).

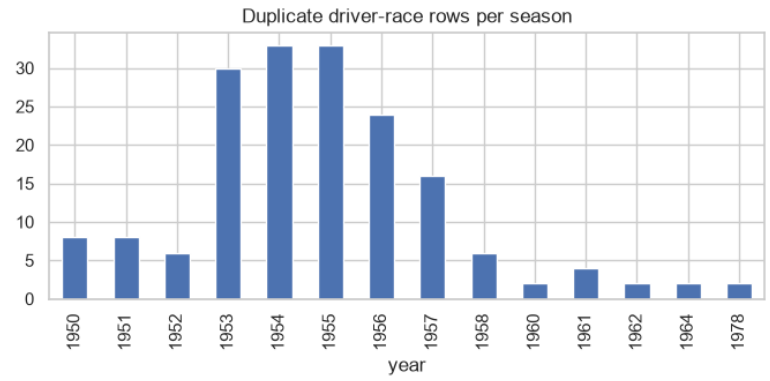


Figure 2. Duplicate (raceId, driverId) rows per season: almost all in the 1950s.

2.4 Rows that are not prediction cases

Step	Rows left	Reason
Raw results.csv	26,759	
Duplicate rule	26,668	One row per driver per race (section 2.3).
Drop the Indianapolis 500 (1950-1960)	26,287	107 drivers, only 3 of whom raced anywhere else: a separate population on an oval.
Drop positionText F and W	24,586	Failed to qualify or withdrew: they never started, which is known after qualifying.
Drop grid = 0 with 0 laps	24,571	Excluded, disqualified or broken before the start: they did not start either.
grid = 0 with laps (72 rows, 2015+)	24,571	Pit-lane starts. Left as 0 they look better than pole, so grid = last grid place of the race + 1.

raceId is not in time order (raceId 1 is a 2009 race), so the rows are sorted by date before any "previous races" feature is built.

2.5 EDA before and after cleaning

Before cleaning, the rows per season spike in the late 1980s and early 1990s and the podium rate drops in the same years, because every race still has about three podiums. Most of those extra rows were cars that never started, so after cleaning the bump is much smaller, the lowest podium rates move up, and the spike at grid 0 is gone (Figures 3-6). Qualifying data starts in 1994, is patchy in 1996-2002, and is complete from 2003 (Figure 7): this is the qualifying-coverage drift that the qualifying features must handle.

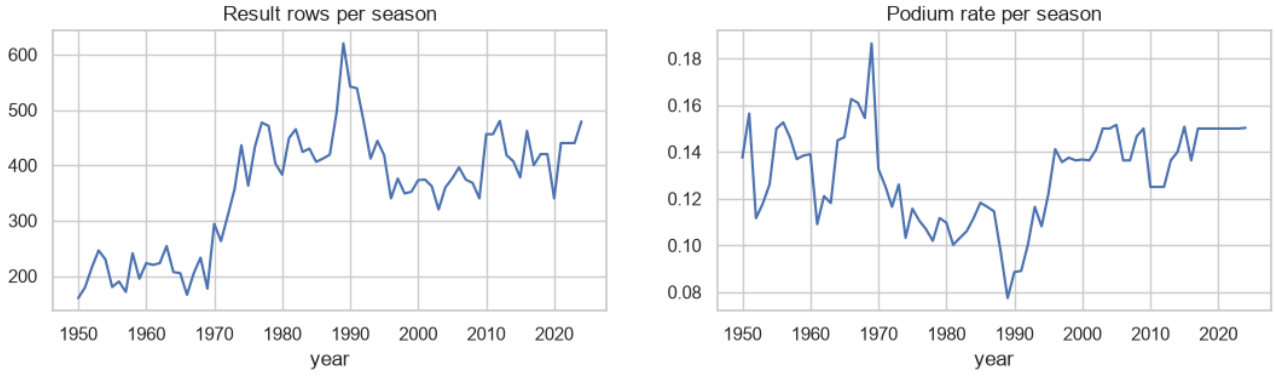


Figure 3. Before cleaning: result rows and podium rate per season.

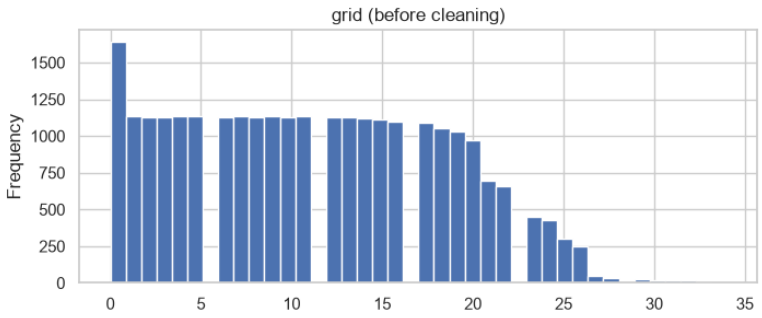


Figure 4. Before cleaning: grid has a spike at 0, which is not a real starting slot.

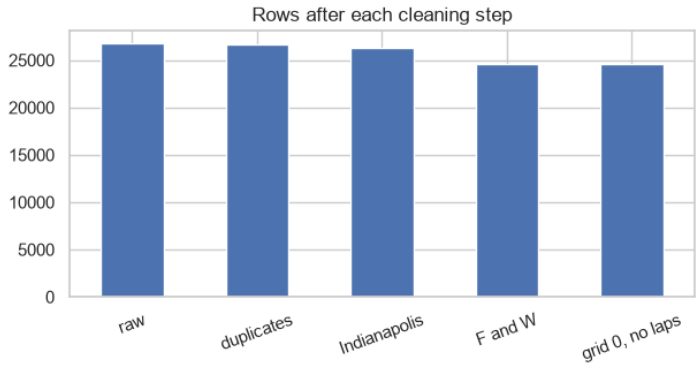


Figure 5. Rows left after each cleaning step.

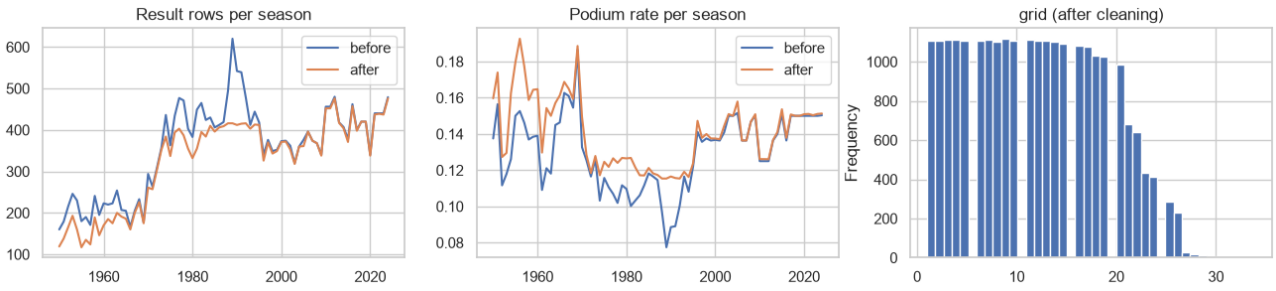


Figure 6. Before vs after cleaning: rows per season, podium rate per season, and the grid distribution after cleaning.

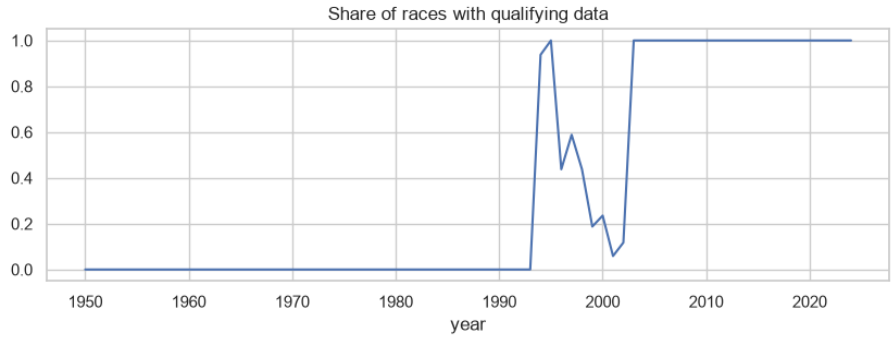


Figure 7. Share of races with any qualifying data, per season.

3. Data-engineering questions

All three answers need multi-hop joins: results.csv alone has neither the year, the circuit, the nationality nor the status text.

Q1. Which circuits convert a front-row start into a podium most often, and does that change after 2014?

Joins: results -> races (raceId) -> circuits (circuitId), and results -> qualifying (raceId, driverId). **Grain:** one front-row starter in one race. **Two definitions:** front row by **grid** (grid 1 or 2, where the car actually started) and by **qualifying** (qualifying position 1 or 2, before penalties). **Years:** 2003-2024, because qualifying is only complete from 2003. That gives two eras of 11 seasons: 2003-2013 and 2014-2024 (hybrid era and later). Circuits need at least 5 races in an era, because with fewer races one result moves the rate by more than 10 points.

Answer. A front-row start became a podium in **68.3%** of cases in 2003-2013 (272/398) and **76.2%** in 2014-2024 (345/453). The top circuits were Magny-Cours (91.7%), Istanbul Park (78.6%), Monza and Monaco (77.3%) before 2014, and Monaco (89.5%), Suzuka (88.9%) and Yas Marina (86.4%) from 2014. **The ranking changed a lot:** for the 14 circuits with enough races in both eras the Spearman rank correlation is only 0.36. Yas Marina (60.0% -> 86.4%) and Interlagos (59.1% -> 78.9%) climbed the most, and Monza fell from the top to 10th of 14.

Grid vs qualifying. Qualifying position and grid differ on 27.5% of starts from 2003, but the front row is stable: of 851 top-2 qualifiers, 36 (4.2%) did not start from the front row, mostly because of penalties. Rates by qualifying are a little lower (67.6% and 74.2%), and the circuit ranking is nearly the same with both definitions (rank correlation 0.89 and 0.88).

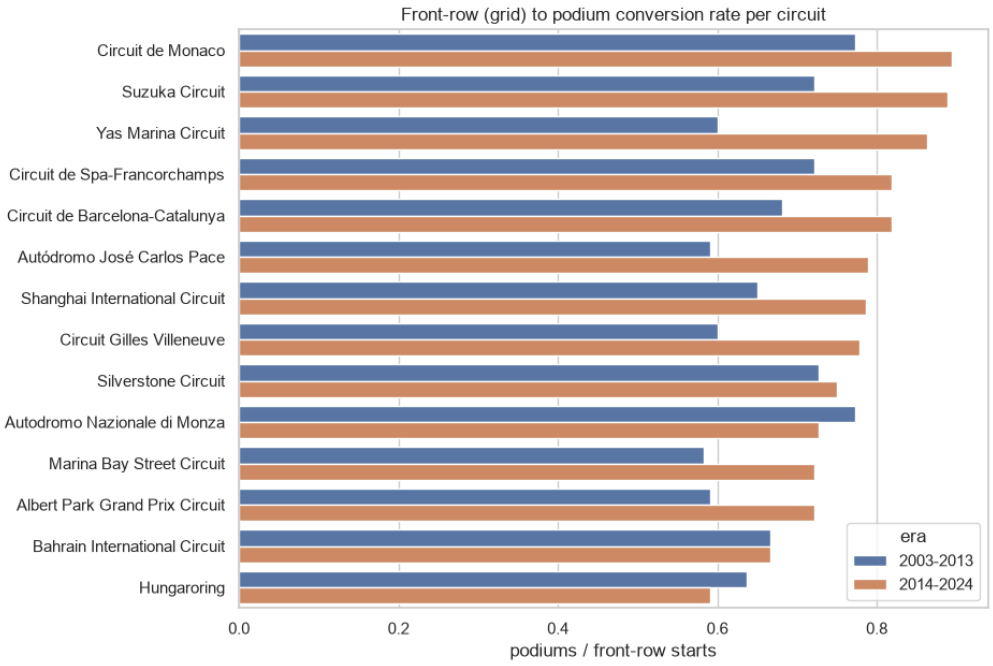


Figure 8. Q1: front-row (grid) to podium conversion rate per circuit and era (circuits with at least 5 races in both eras).

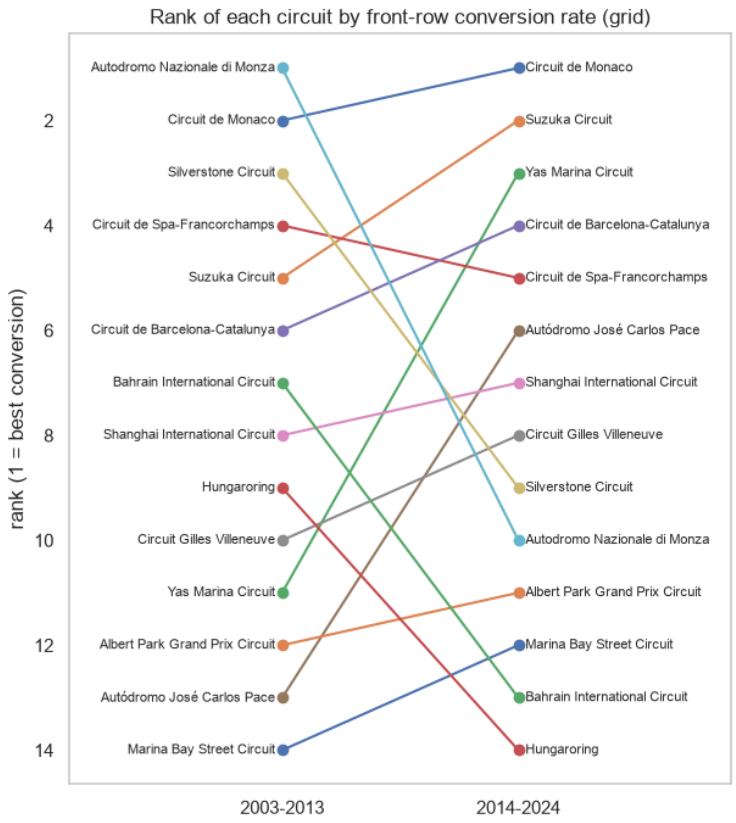


Figure 9. Q1: rank of each circuit in 2003-2013 vs 2014-2024. Crossing lines mean the ranking changed.

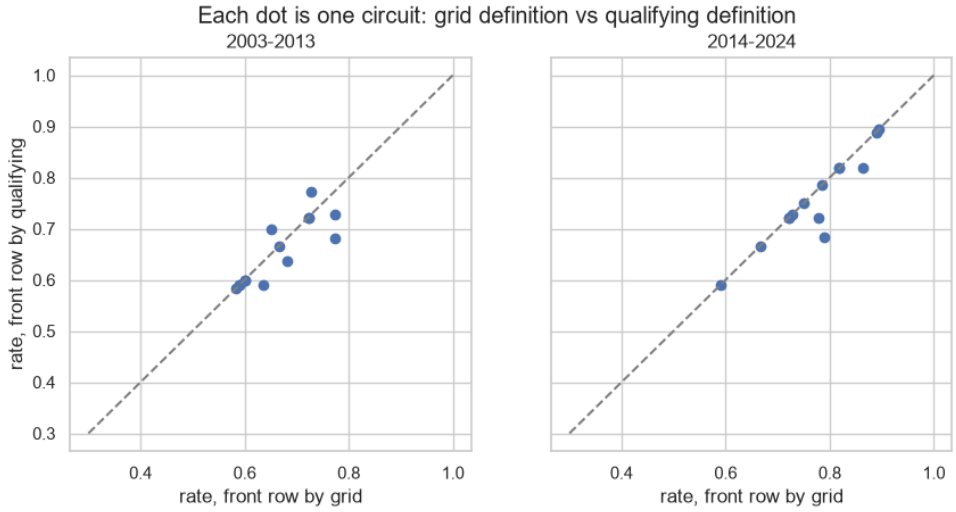


Figure 10. Q1: conversion rate per circuit, grid definition vs qualifying definition (dashed line = equal).

Missingness: 21 starters from 2003 have no qualifying row (kept by a left join, so they cannot be "front row by qualifying"). grid = 0 never counts as front row. Drivers who did not start are dropped, so the 2005 US Grand Prix (grid 1-4 withdrew) has no front-row starter. **Pre-race cutoff:** both front-row flags are known before the start, and the podium is the outcome. Each circuit rate rests on only 10-22 starts, so the order of neighbouring circuits is noisy. The overall rise from 68% to 76% is the solid result.

Q2. Do drivers podium more often at a home circuit, after controlling for starting position?

Joins: results -> races -> circuits (circuit country) and results -> drivers (nationality). **Home** means the circuit country is the country of the driver's nationality, through a documented mapping ("British" -> "UK", "Monegasque" -> "Monaco", "East German" -> "Germany", dual nationalities -> the first one). **Removed:** the Indianapolis 500 (99.7% "home" entries, a separate population), drivers who did not start, grid = 0 starts (no grid group), and 43 drivers whose country never hosted a race (they can never be at home). That leaves 22,175

starts: 1,927 at home and 20,248 away.

Grid group	Home	Away	Difference
1-3	57.5% (131/228)	56.4% (1,548/2,747)	+1.1 points
4-6	27.2% (61/224)	26.1% (704/2,701)	+1.2 points
7-10	7.5% (25/335)	9.1% (333/3,669)	-1.6 points
11+	1.1% (13/1,140)	1.6% (181/11,131)	-0.5 points

Answer: no home advantage once starting position is controlled. Before any control, home drivers reach the podium *less* often (11.9% vs 13.7%), but they also start further back on average (mean grid 12.7 vs 11.8). Inside grid groups the differences go both ways. Their weighted average is -0.3 points (-0.2 points with every grid position as its own group). A logistic regression *podium ~ grid group + home* gives a home odds ratio of **0.96 (95% CI 0.81-1.14, p = 0.65)**. This controls for the grid only, not for the car or the driver, so it is an association, not a causal statement.

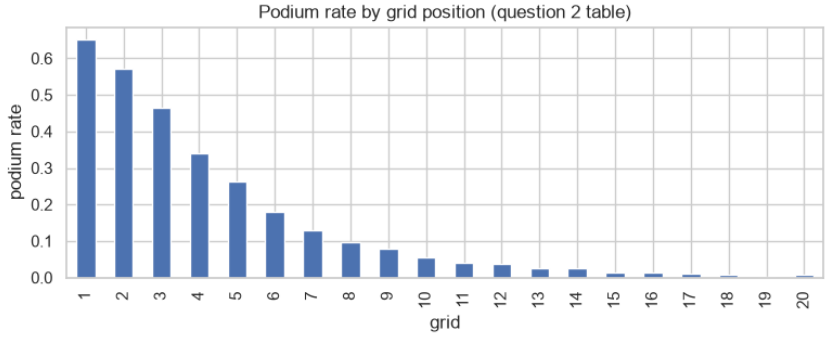


Figure 11. Q2: podium rate by grid position, used to choose the grid groups 1-3, 4-6, 7-10, 11+.

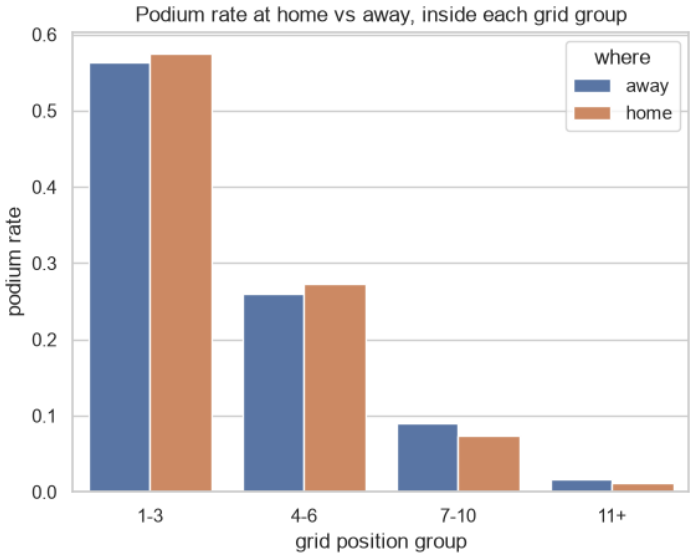


Figure 12. Q2: podium rate at home vs away inside each grid group.

Q3. Which constructors have the highest mechanical-retirement rate in 2014-2021 vs 2022-2024?

Joins: results -> races (year), results -> status (reason the race ended), results -> constructors (team). **Grain:** one car that started one race. **Rate** = mechanical retirements / race starts. Entries that never started (W) are excluded from both the numerator and the denominator, even when the reason was mechanical, because the brief defines the rate over race starts. **Status mapping:** every status seen in 2014-2024 is classified by hand as mechanical (a named part or system failed: Engine, Power Unit, Gearbox, Hydraulics, Brakes, ...), accident (Collision, Accident, Collision damage, Debris), driver (Spun off, Illness), regulation (Disqualified, Excluded), unclear (Retired, Withdrew, Puncture, Tyre) or finished (Finished, +n Laps). **Team lineages:** renamed teams are grouped (Toro Rosso / AlphaTauri / RB, Force India / Racing Point / Aston Martin, Lotus F1 / Renault /

Alpine, Sauber / Alfa Romeo, Marussia / Manor), otherwise the two periods cannot be compared for those teams. Minimum: 50 starts in the period (this removes Caterham, 33 starts).

Rank	2014-2021	Rate	2022-2024	Rate
1	Lotus F1 / Renault / Alpine	14.8% (47/318)	Lotus F1 / Renault / Alpine	10.4% (14/135)
2	Toro Rosso / AlphaTauri / RB	14.4% (46/319)	Sauber / Alfa Romeo	8.8% (12/136)
3	McLaren	13.6% (43/316)	Toro Rosso / AlphaTauri / RB	6.6% (9/136)
4	Red Bull	10.7% (34/318)	Williams	6.0% (8/134)
5	Haas	9.5% (23/241)	Aston Martin lineage, Ferrari, Haas	5.2% (7 each)
last	Mercedes	4.1% (13/320)	McLaren, Red Bull	3.7% (5/136 each)

Answer. Overall, mechanical retirements fell from 9.5% of starts (307/3,245) to 5.9% (80/1,352). **The Enstone team (Lotus F1 / Renault / Alpine) has the highest rate in both periods.** McLaren and Red Bull moved from 3rd and 4th worst to the two most reliable teams. Sauber / Alfa Romeo is the only midfield team that got slightly worse (8.2% -> 8.8%), and Mercedes is about the same (4.1% -> 4.4%). Counting the unclear statuses as mechanical raises every rate a little but keeps the same team on top in both periods. In 2022-2024 each team has only about 135 starts, so teams one or two retirements apart are not really different. **Pre-race cutoff:** status is post-race, so it is only used to describe past races. As a model feature it is turned into a rate over earlier races only (section 4).

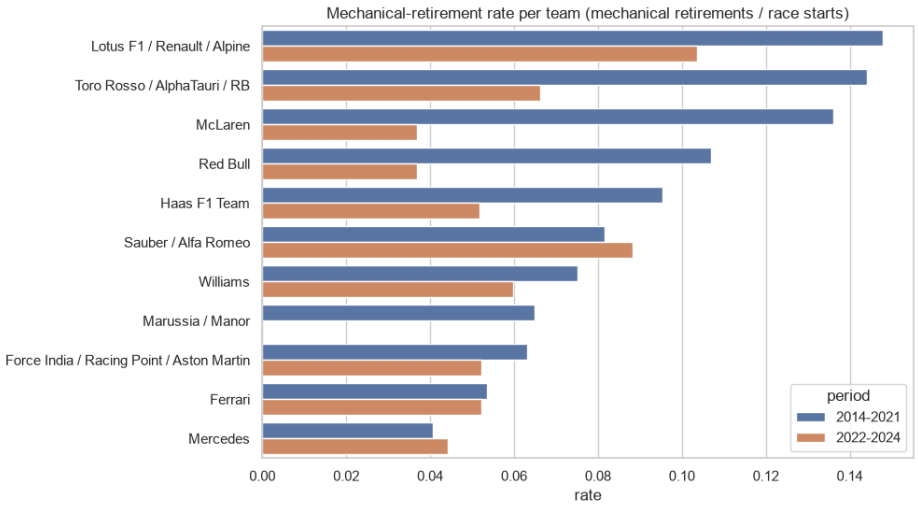


Figure 13. Q3: mechanical-retirement rate per team lineage, sorted by the 2014-2021 rate.

4. Features and why each is valid at the pre-race cutoff

Each feature uses only the starting grid, qualifying, or results of **earlier** races. The key tool is **shift()**: the value of a race is moved one row down before any rolling or cumulative step, so a row never sees its own result. Constructor form is computed on **one row per team per race** first, otherwise one car could see its teammate's result from the same race. The standings tables are post-race, so each row takes the standings of the **previous round of the same season**. Points are divided by the leader's points because a win was worth 8, 9, 10 or 25 points depending on the era.

Group	Features	Why it is known before the race
Grid	grid_position, grid_position_relative (grid / field size), front_row_start	The starting grid is fixed before the start.
Qualifying	grid_penalty_places (grid - qualifying position), qualifying_gap_to_pole_percent, has_qualifying_data	Qualifying is finished before the race. Grid and qualifying position correlate at 0.97, so only their difference is kept.
Driver form	driver_races_before, podium rate last 5, average finish last 5, retirement rate last 10, career podium rate, podium rate at this circuit	Earlier races only (shift). Checked by hand on one driver.
Constructor form	podium rate last 5, average finish last 5, mechanical-retirement rate last 10	Earlier races only, one row per team per race before shifting.
Standings	driver points share, standing position, season wins; constructor points share, standing position	Previous round. No row means 0 points and 0 wins.
Context	driver_age, home_race, season_progress, circuit_podium_rate_for_grid_group	Birth date, nationality and calendar are known in advance. The circuit rate uses earlier races at the circuit only.

Evidence for the choices. The podium rate falls fast and non-linearly with grid position (Figures 14-15), so grid is the core signal, and front_row_start and the circuit-by-grid-group rate help a linear model follow the curve. Among drivers starting 7th-10th, those moved back 3+ places by a penalty reached the podium 21.7% of the time against 5.7% for the others: fast cars out of position, which grid alone cannot see. Recent form works too: the podium rate climbs from about 4% for drivers with no podium in their last 5 races to above 60% for 5 out of 5 (Figure 16). No feature correlates near +-1 with the target (Figure 17), which would be the sign of a leak. **No sprint features:** sprints only exist from 2021, so the column would be empty for every training row, and the format changed almost every year.

Leakage checks in the notebook: the row count is unchanged after all merges, there are no duplicate keys, no post-race column is in the feature list, and one form value recomputed by hand matches the table.

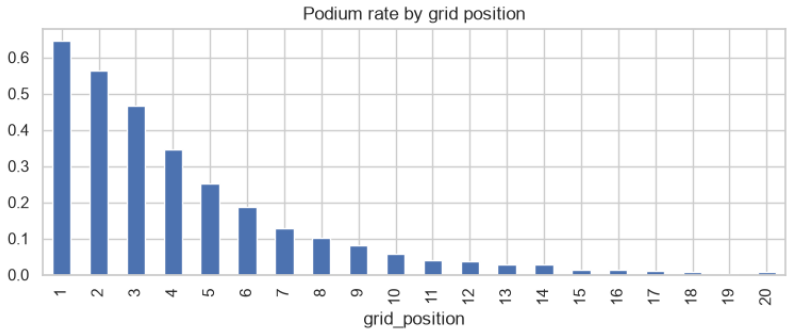


Figure 14. Podium rate by grid position (all rows of the modeling table).

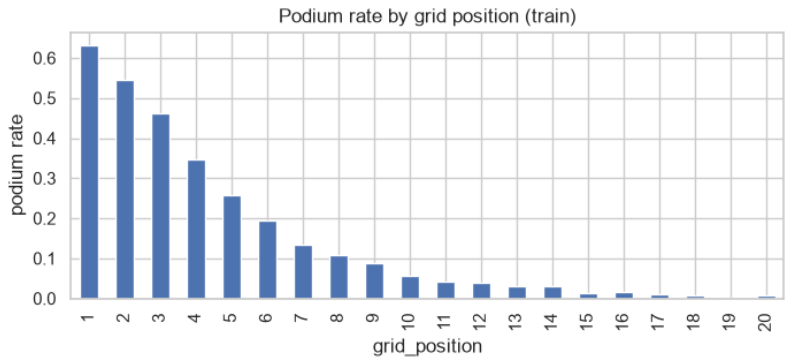


Figure 15. The same on the training rows only, as checked before modeling.

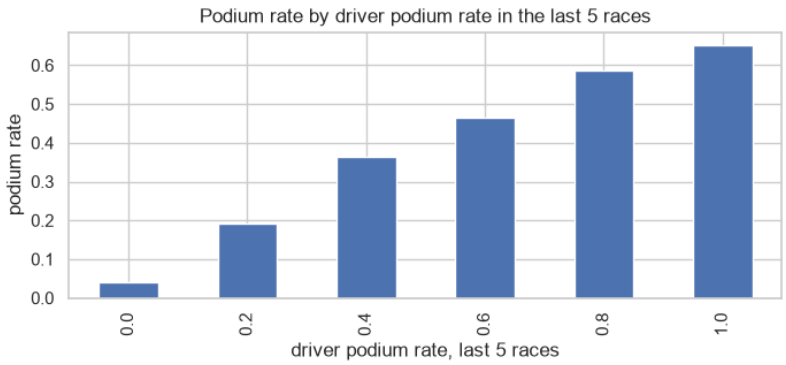


Figure 16. Podium rate by the driver's podium rate in the previous 5 races (drivers with at least 5 earlier races).

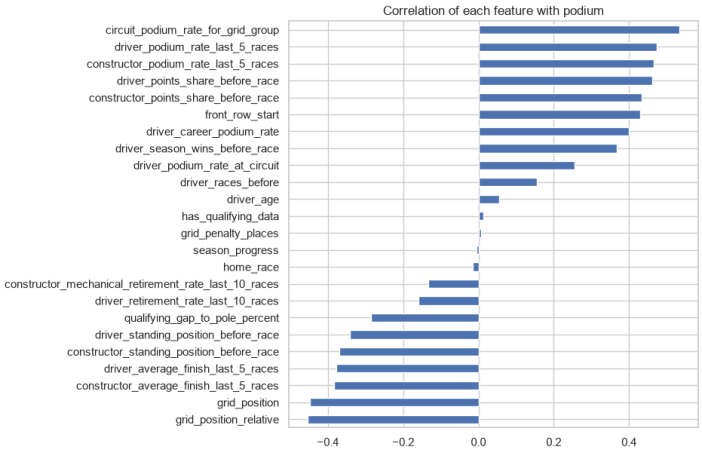


Figure 17. Leakage check: correlation of each feature with the target on all rows. None is close to +-1.

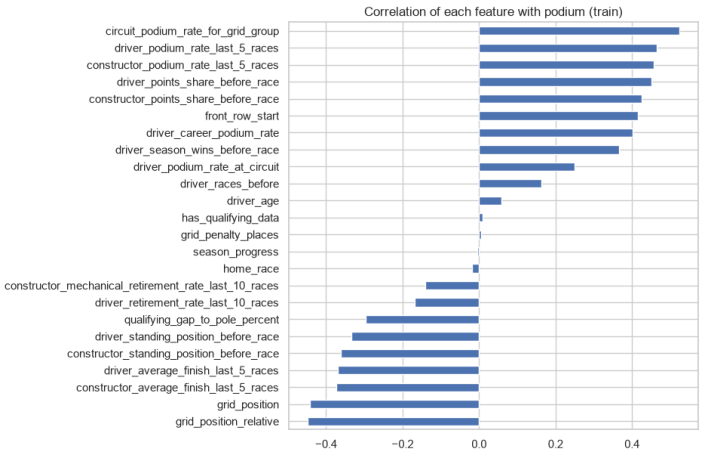


Figure 18. Correlation of each feature with the target (training rows).

The features overlap: four pairs correlate above 0.8 (grid_position and grid_position_relative 0.96, grid_position_relative and the circuit rate -0.81, driver and constructor podium rate 0.81, driver and constructor points share 0.81). This is why removing a single group costs little in the ablation (section 7).

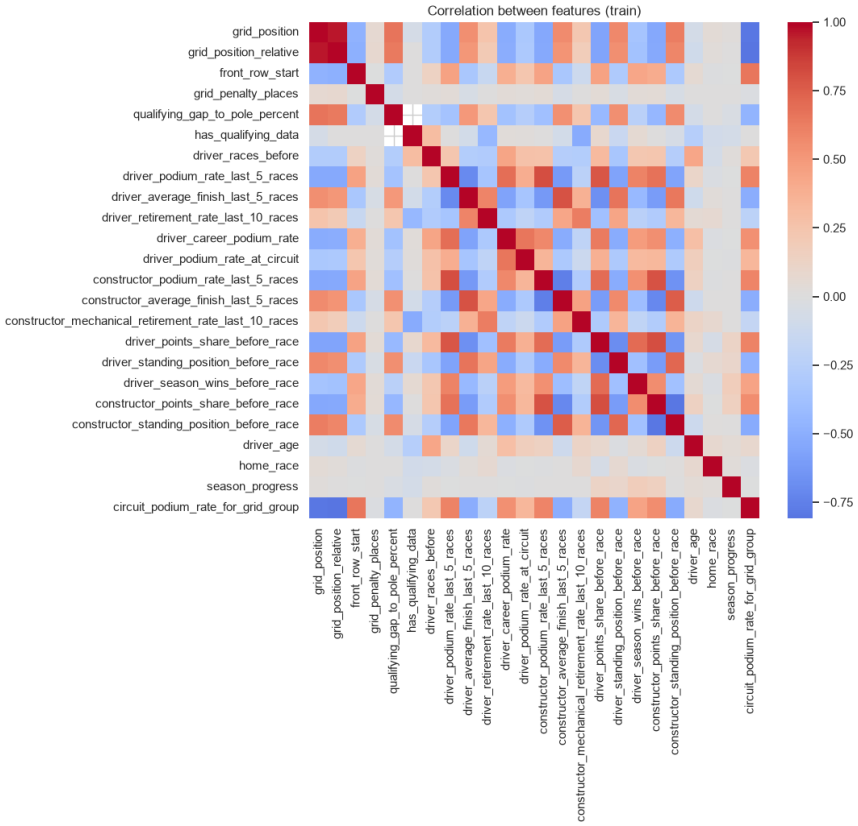


Figure 19. Correlation between the 24 features (training rows).

5. Split and pre-processing

Split by season: train 1950-2019 (22,444 rows, 13.5% podium), validation 2020-2021 (775 rows, 15.1%), test 2022-2024 (1,352 rows, 15.1%). The split is not random because at prediction time only the past is known, and form and standings features link neighbouring races. A random split would leak in both directions. year is used for the split only, never as a feature.

Missing values are filled with the **training median** (several features are skewed). The biggest case is the qualifying gap, missing in 14,284 training rows (no lap times before 2003) but almost complete in validation and test. has_qualifying_data tells the model which races were filled. **Scaling:** standardisation with the training mean and std, for logistic regression and the neural network. Trees get the filled, unscaled table. Every model uses the same rows, the same 24 features and the same filled values. **Effect of scaling:** the ranges were very different (driver_races_before goes up to a few hundred, the rates stay between 0 and 1). After scaling every feature has mean 0 and std 1 on train, and the shape of each distribution is unchanged: only the axis moves.

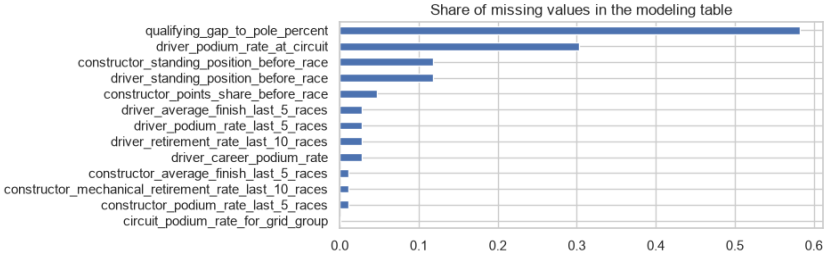


Figure 20. Share of missing values left in the saved modeling table, before filling.

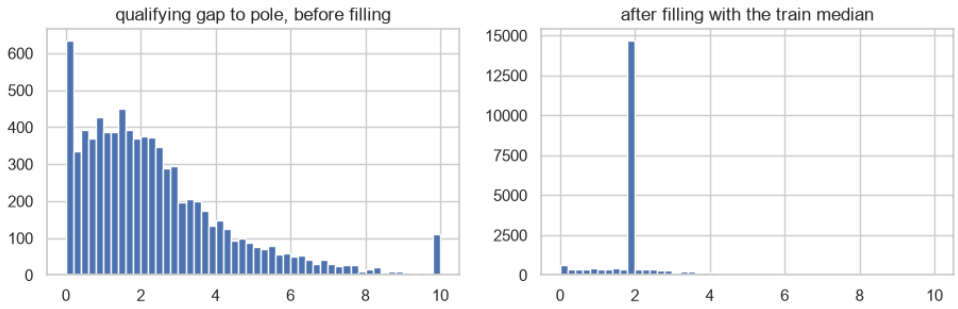


Figure 21. Effect of median filling on the qualifying gap: the filled rows form a spike at the median.

Pre-processing-order experiments (logistic regression, PR-AUC):

Experiment	Our pipeline (val / test)	Alternative (val / test)	Finding
A. Split first vs fill and scale the whole table first	0.7459 / 0.7233	0.7459 / 0.7235	Same score: the medians barely move because val + test are 9% of rows. We still split first, the only leak-free order.
B. Fill then scale vs scale then fill	0.7459 / 0.7233	0.7461 / 0.7236	The std of the qualifying gap changes (1.57 vs 2.57), but a linear model absorbs the scale in its weight.
C. Missing flags before vs after filling	0.7459 / 0.7233	0.7446 / 0.7213	Flags after filling flag 0 cells, before filling 24,120. The flags do not help, so they are not used.

6. Models and metrics

Metrics: ROC-AUC, PR-AUC, F1, precision and recall, for every model. **Primary metric: PR-AUC.** Only about 14% of rows are podiums, and ROC-AUC also rewards the many easy "no podium" rows at the back of the grid, so it is high for every model. PR-AUC only improves when podium rows are ranked better. **Secondary metrics:** ROC-AUC (comparability) and F1 (the final yes/no decision). The F1 threshold of each model is the one with the best F1 on **validation**. All hyperparameters were also chosen on validation. The test set was scored once, at the end.

- **Logistic regression (baseline model):** a weighted sum of the features squeezed into a probability. Each feature pushes the probability in one direction only, and there are no interactions unless they are built by hand. Changing C (0.01-10) moved validation PR-AUC by less than 0.002, so the default C = 1 is kept. *Limits:* it keeps adding or subtracting for every extra unit, so outliers (a huge gap to pole) get extreme values.
- **Random forest:** averages many decorrelated trees and handles curves and interactions. The default (fully grown trees) overfits (train PR-AUC 1.000, validation 0.685), so max_depth = 5 (validation 0.732). *Limits:* deep trees memorise the training rows, and a forest cannot predict outside what it has seen.
- **Shallow neural network (Keras):** 2 hidden layers (32-16) with dropout 0.2. ReLU, sigmoid output, binary cross-entropy, Adam, early stopping on validation loss (patience 10). Retraining with three other seeds moves validation PR-AUC between 0.744 and 0.751, so **differences under about 0.01 are noise**. *Limits:* the result depends on the random start, it needs scaled inputs, and it is harder to explain than a list of weights.

	Validation			Test (unseen)					
Model	ROC AUC	PR AUC	F1	ROC AUC	PR AUC	F1	Precision	Recall	Threshold
Logistic regression	0.931	0.746	0.713	0.936	**0.723**	0.676	0.551	0.873	0.233
Random forest	0.932	0.732	0.716	0.941	0.719	0.669	0.566	0.819	0.272
Neural network (32-16)	0.931	0.751	0.713	0.937	0.715	0.676	0.592	0.789	0.319

Train PR-AUC: logistic regression 0.578, random forest 0.612, neural network 0.600. Validation and test scores are above train for every model. This is not a bug: the training rows go back to 1950, when races were far less predictable, and the validation and test races are modern.

Result. The three models are within the noise of each other. The non-grid features add real information: the same logistic regression on the grid features alone reaches only 0.655 / 0.658 PR-AUC (validation / test, section 7). **Final model: logistic regression**, chosen on validation: it is tied with the best network (0.746 vs 0.751, inside the seed noise), and among tied models we take the simplest. It is deterministic, trains in a second, and

its weights can be read. The test column (PR-AUC 0.723, ROC-AUC 0.936) confirms the choice but did not make it. The complex models did not earn their complexity because the hand-built features (form, standings, circuit rate) already capture the non-linear structure. At the validation threshold of 0.23 the model finds 178 of the 204 test podiums (recall 0.87) with 145 false alarms (precision 0.55), about 5 candidates per race for 3 places.

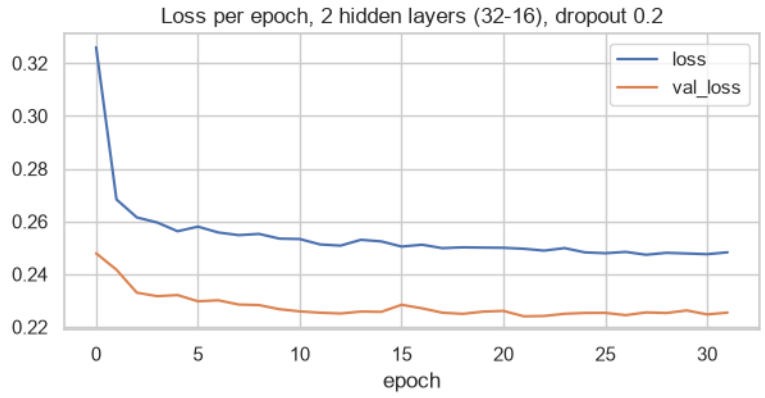


Figure 22. Training and validation loss per epoch of the network (early stopping keeps the best epoch).

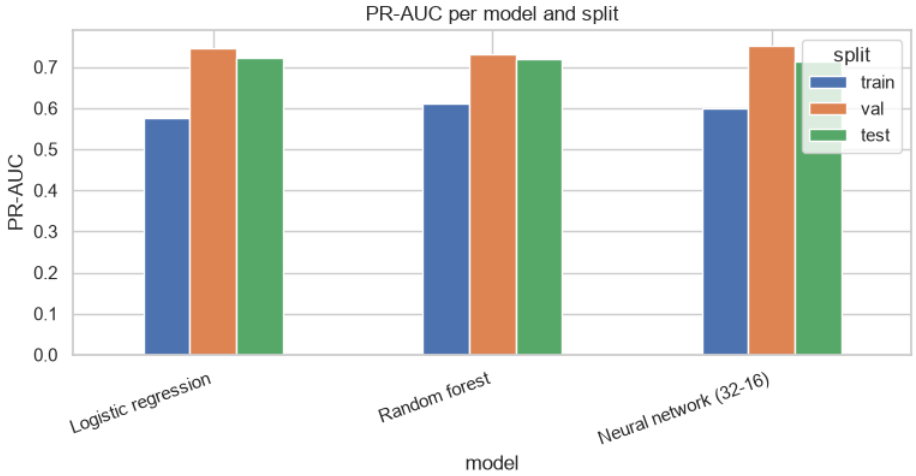


Figure 23. PR-AUC per model on train, validation and test.

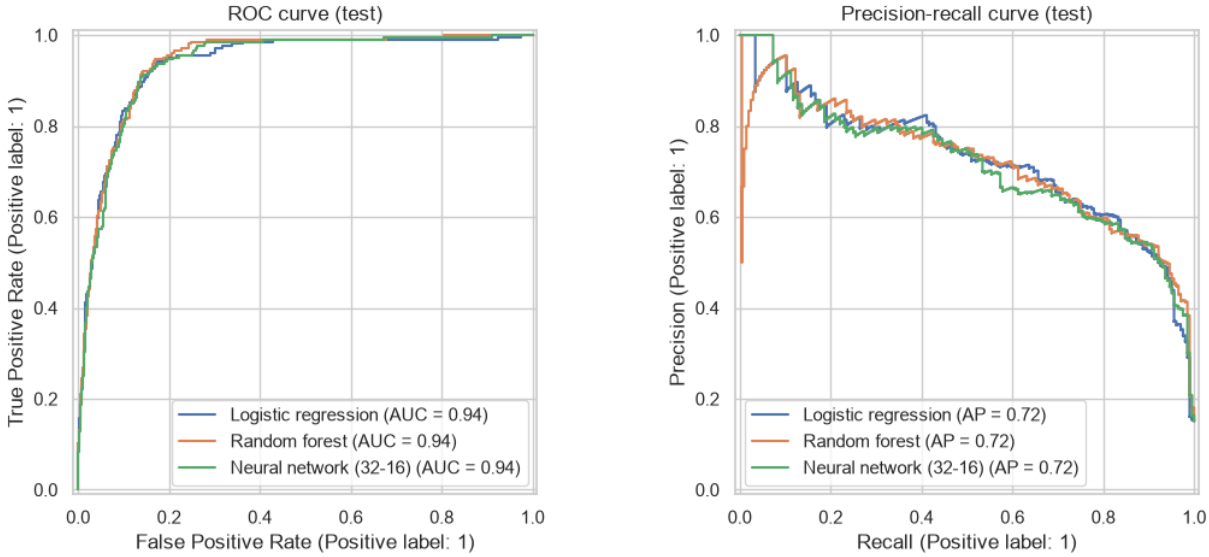


Figure 24. ROC and precision-recall curves on the unseen test set.

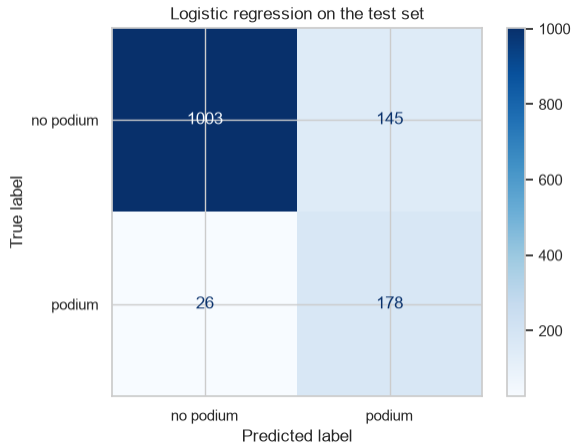


Figure 25. Confusion matrix of the final model on the test set (threshold 0.23 from validation).

7. Experiments: feature-group ablation

One feature group is removed at a time and the logistic regression is retrained. Then the view is turned around: one group alone.

Group removed	Val drop	Test drop	Group alone: val / test PR-AUC
grid	-0.011	+0.022	0.655 / 0.658
qualifying	+0.004	+0.001	0.639 / 0.607
driver form	+0.008	-0.002	0.653 / 0.519
constructor form	-0.009	+0.007	0.534 / 0.509
standings	+0.007	+0.009	0.661 / 0.562
context	+0.013	+0.000	0.635 / 0.574
(all features)			0.746 / 0.723

No single group is essential: every drop is at most about 0.02, and several are negative, all near the noise level of 0.01. The groups overlap (driver and constructor form correlate at 0.81, and the circuit rate is built from the grid), so when one group is removed the others fill in. **Every group alone is far below the full model**, so the gain comes from combining groups. Grid alone is the strongest and most stable group. Standings and driver form alone look good on validation but fall on test. Further experiments: absolute vs relative grid makes no difference (within 0.003), and training from 1994 (10,091 rows) scores the same as training from 1950 (validation 0.752 vs 0.746), so the old seasons do not help.

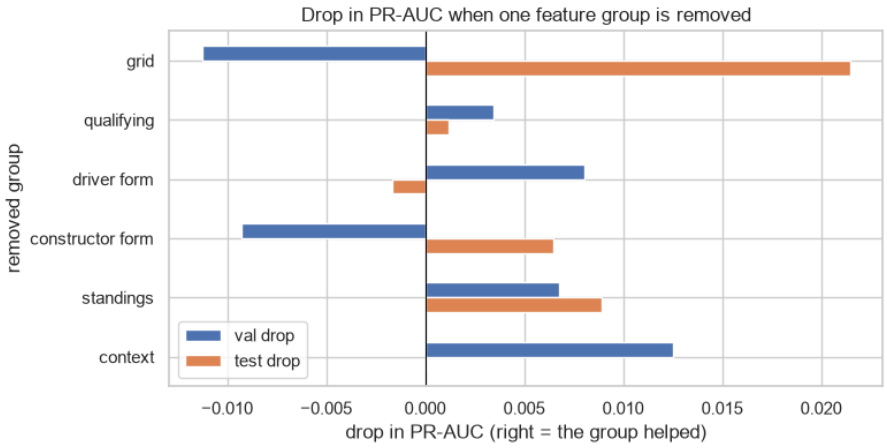


Figure 26. Drop in PR-AUC when one feature group is removed (right = the group helped).

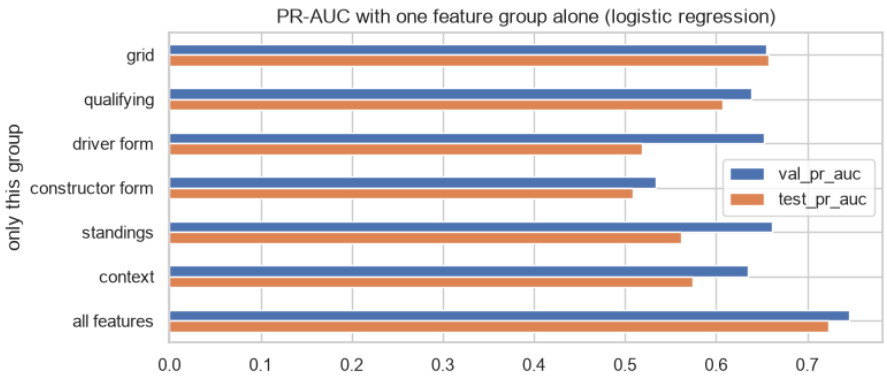


Figure 27. PR-AUC with one feature group alone, against all features.

8. Explainability

Important: these explanations show what the model relies on. They are not causal effects. A feature being important does not prove that it causes podium finishes. For example, the model leans on grid position, but a slow car moved to pole by hand would not become a podium car.

8.1 Global: permutation importance and SHAP

Permutation importance (validation, PR-AUC, 10 shuffles): grid_position stands far above everything else (shuffling it costs about 0.17 PR-AUC). Then come driver_career_podium_rate, circuit_podium_rate_for_grid_group and qualifying_gap_to_pole_percent at about 0.02 each. Most other features are near zero. A zero does not mean useless: when two features carry the same information, shuffling one leaves the other to do the job.

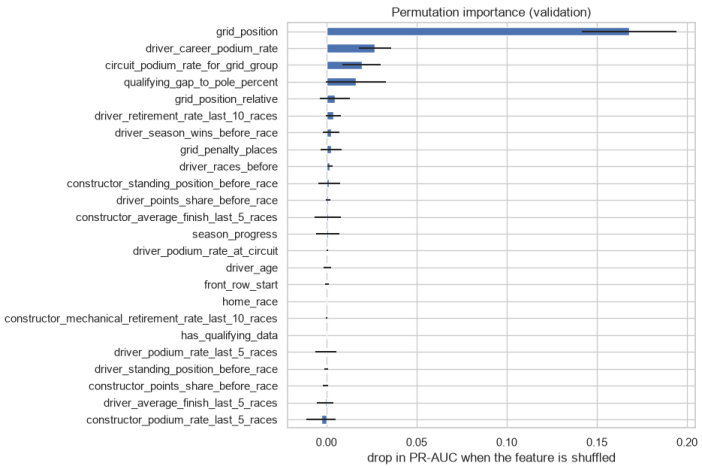


Figure 28. Permutation importance of the final model on the validation set.

SHAP (LinearExplainer, in log-odds) agrees on the top features. In the beeswarm, a high grid number (red) pushes the prediction down and a front start (blue) pushes it up. A few drivers with a huge gap to pole get extreme negative values, because a linear model keeps subtracting for every extra percent: this is a limit of the model. A positive grid_penalty_places pushes up: a driver who starts further back than qualified is faster than the grid slot suggests. has_qualifying_data ranks high in SHAP but is zero in permutation importance. Every validation race has qualifying data, so it shifts every row by the same amount and cannot change the order. SHAP measures the distance from the training average, while permutation importance measures what the ranking needs.



Figure 29. SHAP beeswarm: each dot is one validation row (red = high feature value).

8.2 Local: one prediction (LIME)

The row is the pole sitter of the last race of 2021 (round 22). The model gives 0.85, and the driver did finish on the podium. LIME makes many small random changes to the row, asks the model for each, and fits a small linear model around it. The grid position is the longest green bar, then the gap to pole, the circuit rate and the career podium rate: the same features that lead the global explanations. The conditions are in scaled units because the model works on the scaled table (grid_position <= -0.87 means a start near the front). This explains why the model said 0.85. It does not say these features caused the podium.

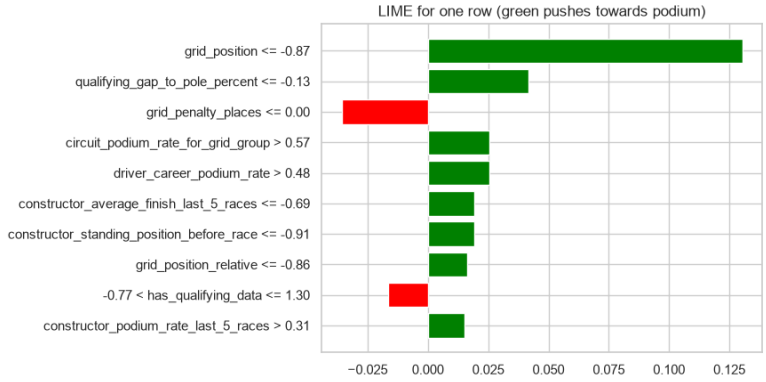


Figure 30. LIME explanation of the same prediction (green pushes towards podium).

9. Inference function

predict_podium(record) takes the pre-race features of one driver in one race as a dict. It repeats the training pipeline with values learned on training rows only: it fills missing features with the training medians, scales with the training mean and std, predicts with the logistic regression, and compares with the validation threshold. A feature absent from the record is treated as missing. On a complete test record it returns (0.358, podium). With three values removed (qualifying gap, podium rate at the circuit, standing position) it returns (0.276, podium). The prediction still works, but it is less reliable, because a typical value replaced the driver's real circuit podium rate of 1.0.

10. Known limitations

- **Qualifying-coverage drift:** qualifying lap times are missing for most races before 2003 (58% of all rows for the gap feature). The qualifying features are mostly filled values in training but complete in validation and test.
- **Era sensitivity:** most training rows come from a different kind of F1 (bigger fields, lower reliability, other points systems). Training from 1994 scores the same with less than half the rows.

- **Small validation set:** 775 rows and 117 podiums, so differences under about 0.01 PR-AUC (the seed noise of the network) cannot be separated.
- **Team renames reset history:** constructor form follows constructorId, so a renamed team starts with no history.
- **Shared drives:** the kept duplicate row can carry the grid slot of the car the driver took over.
- **season_progress in 2020:** the COVID calendar was built during the season, so the number of rounds was not fully known in advance.
- **Associations, not causes:** the data-engineering answers and the explanations describe patterns in the data and in the model, not causal effects.